

Guía de instalación: openwiki + hub unificado sobre MiniMax

Guía de instalación y uso

Objetivo: dejar `openwiki` (CLI de langchain-ai) funcionando con el modelo `MiniMax-M3` como proveedor, más un **pre-commit hook global** que regenera la wiki automáticamente al hacer commit, y un **hub web** que muestra todas las wikis del usuario en una sola URL (`http://localhost:8765`).

Probado en macOS con `zsh` . Funciona igual en Linux cambiando las rutas de `~/.zshrc` por `~/.bashrc` .

0. Prerrequisitos

```
node --version    # >= 18
npm --version     # >= 9
git --version     # >= 2
python3 --version # >= 3.9
```

Si falta alguno, en macOS:

```
brew install node git python3
```

1. Instalar `openwiki`

```
npm install -g openwiki
openwiki --version    # debe imprimir 0.0.x
```

Si usas `pnpm` / `bun` en lugar de `npm` , instálalo con tu gestor (`pnpm add -g openwiki`).

2. Configurar el proveedor MiniMax

`openwiki` soporta Anthropic-compatible out-of-the-box. Como `@langchain/anthropic` envía `x-api-key` (no `Authorization: Bearer`), encaja tal cual con el endpoint `/anthropic` de MiniMax.

2.1 Crear `~/.openwiki/.env`

```
mkdir -p ~/.openwiki
chmod 700 ~/.openwiki
cat > ~/.openwiki/.env <<'EOF'
OPENWIKI_PROVIDER="anthropic"
ANTHROPIC_API_KEY="<PEGAR_AQUI_TU_TOKEN_MINIMAX_sk-cp-...>"
ANTHROPIC_BASE_URL="https://api.minimax.io/anthropic"
OPENWIKI_MODEL_ID="MiniMax-M3"
EOF
chmod 600 ~/.openwiki/.env
```

Importante: sustituye `<PEGAR_AQUI_TU_TOKEN_MINIMAX_sk-cp-...>` por tu token real de platform.minimax.io. El prefijo `sk-cp-` es el típico del plan MiniMax token-plan.

2.2 Verificar la conexión

```
mkdir -p /tmp/openwiki-test && cd /tmp/openwiki-test
git init -q
echo "# test" > README.md
openwiki --init -p "Initialise docs" 2>&1 | tail -5
ls openwiki/      # debe haber quickstart.md
```

Si responde y aparece `openwiki/quickstart.md`, la config está bien. Limpia:

```
mavis-trash /tmp/openwiki-test  # o: rm -rf /tmp/openwiki-test
```

3. Pre-commit hook global (sin auto-add)

Esto hace que cada `git commit` en cualquier proyecto nuevo regenere automáticamente `openwiki/`, pero **sin añadir los cambios al commit** (el usuario decide si incluirlos).

```
mkdir -p ~/.git-templates/hooks
cat > ~/.git-templates/hooks/pre-commit <<'EOF'
#!/usr/bin/env bash
# Auto-regenera la wiki local antes de cada commit.
# NO añade los cambios al commit – tú decides si los incluyes.
# Si openwiki falla (API caída, sin credenciales, etc) no bloquea tu commit.
openwiki --update -p "Update wiki with staged changes" --modelId MiniMax-M3 2>/dev/null
EOF
chmod +x ~/.git-templates/hooks/pre-commit

git config --global init.templateDir ~/.git-templates
```

Verificar:

```
mkdir -p /tmp/hook-verify && cd /tmp/hook-verify
rm -rf .git && git init -q
echo "x" > README.md
git add . && git commit -m "first" 2>&1 | tail -3
# Debe imprimir el commit sin errores. Tras el commit, `git status` debe
# mostrar openwiki/ modificada (si la hay), NO staged.
```

4. Construir el hub unificado

El hub es un sitio estático servido con `python3 -m http.server`. Escanea todas las carpetas `openwiki/` del usuario y las expone en `http://localhost:8765` con renderizado de markdown + diagramas Mermaid.

4.1 Crear la estructura

```
mkdir -p ~/openwiki-hub/wikis
```

4.2 Crear `~/openwiki-hub/index.html`

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>OpenWiki Hub</title>
<link rel="stylesheet" href="styles.css" />
<script defer src="https://cdn.jsdelivr.net/npm/marked@13.0.3/marked.min.js"></script>
<script defer src="https://cdn.jsdelivr.net/npm/mermaid@10.9.1/dist/mermaid.min.js"></script>
<script defer src="app.js"></script>
</head>
<body>
<aside id="sidebar">
  <header>
    <h1>📖 OpenWiki Hub</h1>
    <p id="stats"></p>
  </header>
  <nav id="wikis"></nav>
  <footer>
    <small>⚠ corre <code>owhu</code> para refrescar</small>
    <small id="last-build"></small>
  </footer>
</aside>
<main id="content">
  <div class="placeholder">
    <h2>Pick a wiki from the left</h2>
    <p>If the list is empty, run <code>owhu</code> in your terminal.</p>
  </div>
</main>
</body>
</html>
```

4.3 Crear `~/openwiki-hub/styles.css`

```
:root {
  --bg: #0f1115;
  --bg-soft: #161922;
  --bg-soft-2: #1d2130;
  --fg: #e4e6eb;
  --fg-dim: #8b91a3;
  --accent: #7aa2f7;
  --border: #2a2f3d;
  --code-bg: #1a1d27;
  --link: #9eb6ff;
  --link-hover: #c0d2ff;
}

* { box-sizing: border-box; }

html, body {
  margin: 0;
  padding: 0;
  height: 100%;
  font-family: -apple-system, BlinkMacSystemFont, "SF Pro Text", "Segoe UI",
    Helvetica, Arial, sans-serif;
  background: var(--bg);
  color: var(--fg);
  font-size: 15px;
  line-height: 1.55;
}

body {
  display: grid;
  grid-template-columns: 300px 1fr;
  height: 100vh;
}

/* — Sidebar ————— */

#sidebar {
  background: var(--bg-soft);
  border-right: 1px solid var(--border);
  display: flex;
  flex-direction: column;
  overflow: hidden;
}

#sidebar header {
  padding: 18px 18px 12px;
  border-bottom: 1px solid var(--border);
}

#sidebar h1 {
  margin: 0;
  font-size: 17px;
}
```

```
    font-weight: 600;
    letter-spacing: -0.01em;
}

#stats {
    margin: 6px 0 0;
    color: var(--fg-dim);
    font-size: 12px;
}

#wikis {
    flex: 1;
    overflow-y: auto;
    padding: 8px 0;
}

.wiki-group { margin: 4px 0 12px; }

.wiki-group-name {
    padding: 6px 18px 4px;
    font-size: 11px;
    font-weight: 600;
    text-transform: uppercase;
    letter-spacing: 0.06em;
    color: var(--fg-dim);
}

.wiki {
    display: block;
    padding: 7px 18px 7px 22px;
    color: var(--fg);
    text-decoration: none;
    border-left: 2px solid transparent;
    font-size: 13px;
    cursor: pointer;
    user-select: none;
}

.wiki:hover {
    background: var(--bg-soft-2);
    color: var(--link-hover);
}

.wiki.active {
    background: var(--bg-soft-2);
    border-left-color: var(--accent);
    color: var(--accent);
}

.wiki .meta {
    display: block;
    font-size: 11px;
    color: var(--fg-dim);
    margin-top: 1px;
```

```
}

.wiki-files {
  margin-left: 22px;
  border-left: 1px solid var(--border);
}

.wiki-file {
  display: block;
  padding: 4px 14px;
  color: var(--fg-dim);
  font-size: 12px;
  text-decoration: none;
  cursor: pointer;
}

.wiki-file:hover { color: var(--fg); }
.wiki-file.active { color: var(--accent); }

#sidebar footer {
  padding: 12px 18px;
  border-top: 1px solid var(--border);
  display: flex;
  flex-direction: column;
  gap: 6px;
}

#sidebar footer code {
  background: var(--code-bg);
  padding: 1px 5px;
  border-radius: 3px;
}

#last-build {
  color: var(--fg-dim);
  font-size: 10px;
}

/* — Main content ————— */

#content {
  overflow-y: auto;
  padding: 32px 48px 80px;
  max-width: 920px;
}

.placeholder {
  color: var(--fg-dim);
  margin-top: 80px;
  text-align: center;
}

.placeholder code {
  background: var(--code-bg);
```



```
padding: 2px 6px;
border-radius: 4px;
font-size: 13px;
}

.breadcrumb {
  font-size: 12px;
  color: var(--fg-dim);
  margin-bottom: 18px;
}

.breadcrumb a {
  color: var(--fg-dim);
  text-decoration: none;
}

.breadcrumb a:hover { color: var(--accent); }

/* — Markdown typography ————— */

#content h1, #content h2, #content h3, #content h4 {
  font-weight: 600;
  letter-spacing: -0.015em;
  margin-top: 1.6em;
  margin-bottom: 0.6em;
  line-height: 1.25;
}

#content h1 {
  font-size: 28px;
  margin-top: 0.2em;
  padding-bottom: 12px;
  border-bottom: 1px solid var(--border);
}

#content h2 { font-size: 21px; }
#content h3 { font-size: 17px; color: var(--fg); }
#content h4 { font-size: 15px; color: var(--fg-dim); }

#content p {
  margin: 0.7em 0;
  color: #d4d7df;
}

#content a { color: var(--link); text-decoration: none; }
#content a:hover {
  color: var(--link-hover);
  text-decoration: underline;
}

#content ul, #content ol {
  padding-left: 24px;
  margin: 0.6em 0;
}
```

```
#content li {
  margin: 4px 0;
  color: #d4d7df;
}

#content code {
  background: var(--code-bg);
  padding: 2px 6px;
  border-radius: 4px;
  font-family: ui-monospace, "SF Mono", Menlo, Monaco, "Cascadia Mono", monospace;
  font-size: 0.88em;
  color: #f0c674;
}

#content pre {
  background: var(--code-bg);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 14px 16px;
  overflow-x: auto;
  line-height: 1.5;
}

#content pre code {
  background: none;
  padding: 0;
  color: var(--fg);
  font-size: 13px;
}

#content blockquote {
  margin: 1em 0;
  padding: 6px 16px;
  border-left: 3px solid var(--border);
  color: var(--fg-dim);
}

#content table {
  border-collapse: collapse;
  margin: 1em 0;
  width: 100%;
  font-size: 13px;
}

#content th, #content td {
  border: 1px solid var(--border);
  padding: 8px 12px;
  text-align: left;
}

#content th {
  background: var(--bg-soft);
  font-weight: 600;
}
```

```

}

#content hr {
  border: none;
  border-top: 1px solid var(--border);
  margin: 2em 0;
}

#content img {
  max-width: 100%;
  border-radius: 6px;
}

/* — Mermaid diagrams ————— */

.mermaid-diagram {
  margin: 1.4em 0;
  padding: 18px;
  background: var(--bg-soft);
  border: 1px solid var(--border);
  border-radius: 8px;
  overflow-x: auto;
  text-align: center;
}

.mermaid-diagram svg {
  max-width: 100%;
  height: auto;
  display: inline-block;
}

.mermaid-error {
  margin: 1.4em 0;
  padding: 12px 16px;
  background: #3a1f1f;
  border: 1px solid #6b2c2c;
  border-radius: 8px;
  color: #fca5a5;
  font-family: ui-monospace, "SF Mono", Menlo, monospace;
  font-size: 13px;
  white-space: pre-wrap;
}

::-webkit-scrollbar { width: 10px; height: 10px; }
::-webkit-scrollbar-track { background: transparent; }
::-webkit-scrollbar-thumb { background: #2a2f3d; border-radius: 5px; }
::-webkit-scrollbar-thumb:hover { background: #3a4154; }

```

4.4 Crear `~/openwiki-hub/app.js`

```
/* OpenWiki Hub – reads wikis.json and renders markdown files. */

const $ = (sel) => document.querySelector(sel);

let manifest = null;
let activeWiki = null;
let activeFile = null;

async function loadManifest() {
  const res = await fetch("/wikis.json");
  if (!res.ok) throw new Error(`manifest fetch failed (${res.status}). Run 'owhu' first`);
  manifest = await res.json();
  renderSidebar();
}

function renderSidebar() {
  const nav = $("#wikis");
  const stats = $("#stats");
  nav.innerHTML = "";

  const wikiCount = manifest.wikis.length;
  const fileCount = manifest.wikis.reduce((n, w) => n + w.files.length, 0);
  stats.textContent = `${wikiCount} project${wikiCount === 1 ? "" : "s"} · ${fileCount} files`;

  if (wikiCount === 0) {
    nav.innerHTML = `<div style="padding:14px 18px;color:var(--fg-dim);font-size:12px;">
      No wikis yet.<br/><br/>
      Run <code style="background:var(--code-bg);padding:2px 6px;border-radius:4px;">owhu</code> to get started.
    </div>`;
    return;
  }

  for (const wiki of manifest.wikis) {
    const group = document.createElement("div");
    group.className = "wiki-group";

    const groupName = document.createElement("div");
    groupName.className = "wiki-group-name";
    groupName.textContent = wiki.name;
    group.appendChild(groupName);

    const link = document.createElement("a");
    link.className = "wiki";
    link.innerHTML = `${wiki.label}<span class="meta">${wiki.files.length} files</span>`;
    link.onclick = (e) => {
      e.preventDefault();
      openWiki(wiki, wiki.files[0]);
    };
    group.appendChild(link);

    const fileList = document.createElement("div");
```

```

    fileList.className = "wiki-files";
    fileList.style.display = "none";
    for (const file of wiki.files) {
        const f = document.createElement("a");
        f.className = "wiki-file";
        f.textContent = file.label;
        f.dataset.path = file.path;
        f.onclick = (e) => {
            e.preventDefault();
            openWiki(wiki, file);
        };
        fileList.appendChild(f);
    }
    link.after(fileList);
    link._fileList = fileList;

    nav.appendChild(group);
}

$("#last-build").textContent = `Built ${new Date(manifest.builtAt).toLocaleString()}`
}

async function openWiki(wiki, file) {
    if (!file) return;

    document.querySelectorAll(".wiki").forEach((el) => el.classList.remove("active"));
    document.querySelectorAll(".wiki-file").forEach((el) => el.classList.remove("active"));
    const allLinks = document.querySelectorAll(".wiki");
    const idx = manifest.wikis.findIndex((w) => w.name === wiki.name);
    if (idx >= 0 && allLinks[idx]) {
        allLinks[idx].classList.add("active");
        allLinks[idx]._fileList.style.display = "block";
    }
    document.querySelectorAll(".wiki-file").forEach((el) => {
        if (el.dataset.path === file.path) el.classList.add("active");
    });

    activeWiki = wiki;
    activeFile = file;

    const content = $("#content");
    content.innerHTML = `<div class="placeholder">Loading...</div>`;

    try {
        const res = await fetch(`/${file.path}`);
        if (!res.ok) throw new Error(`fetch ${file.path}: ${res.status}`);
        const md = await res.text();
        content.innerHTML = `
            <div class="breadcrumb">
                <a href="#" onclick="event.preventDefault()">${wiki.label}</a>
                &nbsp;/&nbsp;&nbsp; ${file.label}
            </div>
            ${marked.parse(md)}
        `;
    }

```

```

    await renderMermaidBlocks(content);
  } catch (err) {
    content.innerHTML = `<div class="placeholder">Error: ${err.message}</div>`;
  }
}

async function renderMermaidBlocks(root) {
  if (typeof mermaid === "undefined") {
    await new Promise((r) => setTimeout(r, 200));
    if (typeof mermaid === "undefined") {
      console.warn("mermaid.js not loaded");
      return;
    }
  }
  if (!window._mermaidInit) {
    mermaid.initialize({
      startOnLoad: false,
      theme: "dark",
      securityLevel: "loose",
      themeVariables: {
        background: "#161922",
        primaryColor: "#1d2130",
        primaryTextColor: "#e4e6eb",
        primaryBorderColor: "#2a2f3d",
        lineColor: "#8b91a3",
        secondaryColor: "#1d2130",
        tertiaryColor: "#1d2130",
      },
    });
    window._mermaidInit = true;
  }
  const blocks = root.querySelectorAll("pre code.language-mermaid");
  for (const block of blocks) {
    const source = block.textContent;
    const id = "mermaid-" + Math.random().toString(36).slice(2, 10);
    try {
      const { svg } = await mermaid.render(id, source);
      const wrapper = document.createElement("div");
      wrapper.className = "mermaid-diagram";
      wrapper.innerHTML = svg;
      block.parentElement.replaceWith(wrapper);
    } catch (err) {
      const errBox = document.createElement("div");
      errBox.className = "mermaid-error";
      errBox.textContent = `Mermaid error: ${err.message || err}`;
      block.parentElement.replaceWith(errBox);
    }
  }
}

document.addEventListener("DOMContentLoaded", async () => {
  try {
    await loadManifest();
  } catch (e) {

```

```
$("#wikis").innerHTML = `<div style="padding:18px;color:#f87171;font-size:12px;">$  
  }  
  });
```

4.5 Crear `~/openwiki-hub/link.sh`

Este script escanea directorios comunes del usuario buscando `openwiki/`, crea symlinks en `~/openwiki-hub/wikis/`, y escribe un manifest estático (`wikis.json`) que la UI lee.

```
#!/usr/bin/env bash
# Escanea directorios comunes y crea symlinks ~/openwiki-hub/wikis/<name> → <openwiki>
# Escribe ~/openwiki-hub/wikis.json con el manifest.
set -e

HUB="$HOME/openwiki-hub"
WIKIS_DIR="$HUB/wikis"
MANIFEST="$HUB/wikis.json"

ROOTS=(
    "$HOME/Proyectos" "$HOME/projects" "$HOME/code" "$HOME/dev"
    "$HOME/work" "$HOME/repos" "$HOME/src" "$HOME/Documents"
    "$HOME/Documents/GitHub" "$HOME/ai-brain"
)

# 1) Limpiar symlinks viejos
rm -f "$WIKIS_DIR"/*

# 2) Buscar openwiki/ a max 4 niveles
FOUND=0
for root in "${ROOTS[@]"; do
    [[ -d "$root" ]] || continue
    while IFS= read -r -d '' owp; do
        name=$(basename "$(dirname "$owp")")
        slug=$(echo "$name" | tr '[:upper:]' '[:lower:]' | sed -E 's/[^a-z0-9._-]+/-/g')
        [[ -z "$slug" ]] && slug="wiki"
        suffix=""
        while [[ -e "$WIKIS_DIR/${slug}${suffix}" ]]; do suffix="-${RANDOM:0:4}"; done
        ln -s "$owp" "$WIKIS_DIR/${slug}${suffix}"
        FOUND=$((FOUND+1))
    done < <(find "$root" -maxdepth 4 -type d -name openwiki \
        -not -path '*/node_modules/*' -not -path '*/.git/*' \
        -print0 2>/dev/null)
done

# 3) Generar wikis.json
python3 - "$WIKIS_DIR" "$MANIFEST" <<'PYEOF'
import json, sys
from pathlib import Path
from datetime import datetime

wikis_dir = Path(sys.argv[1])
manifest = Path(sys.argv[2])
wikis = []
for entry in sorted(wikis_dir.iterdir()):
    if entry.name.startswith('.'):
        continue
    target = entry.resolve()
    if not target.is_dir():
        continue
    files = []
    for f in sorted(target.iterdir()):
        if f.suffix == ".md" and f.is_file():

```



```

        rel_path = f"wikis/{entry.name}/{f.name}"
        files.append({
            "name": f.name,
            "label": f.stem,
            "path": rel_path,
            "updatedAt": datetime.fromtimestamp(f.stat().st_mtime).strftime("%Y-%m-%d %H:%M:%S")
        })
    wiki_updated = "nunca"
    meta = target / ".last-update.json"
    if meta.exists():
        try:
            with meta.open() as fp:
                wiki_updated = json.load(fp).get("updatedAt", wiki_updated)[:10]
        except Exception:
            pass
    wikis.append({
        "name": entry.name,
        "label": entry.name.replace("-", " ").replace("_", " ").title(),
        "files": files,
        "updatedAt": wiki_updated,
    })

manifest.write_text(json.dumps({
    "wikis": wikis,
    "builtAt": datetime.utcnow().isoformat() + "Z",
}, indent=2))
print(f"📁 {len(wikis)} wikis · manifest → {manifest}")
PYEOF

```

Hacer ejecutable:

```
chmod +x ~/openwiki-hub/link.sh
```

Personalización de `RROOTS` : si guardas tus proyectos en otros directorios (p.ej. `~/Code` , `~/Trabajo` , `~/workspace`), añade esas rutas al array `RROOTS=( ... )` dentro de `link.sh` .

5. Aliases en `~/.zshrc`

Añadir al final de `~/.zshrc` :

```

# — openwiki (langchain-ai/openwiki) —————
# Config: ~/.openwiki/.env (Anthropic-compatible, MiniMax)
alias ow='openwiki'
alias owi='openwiki --init -p "Documenta el repo desde cero" --modelId MiniMax-M3'
alias owu='openwiki --update -p "Actualiza la wiki con los cambios recientes" --modelId

# — openwiki hub (visor unificado) —————
alias owhu='_owhu_run'
alias owhu-link='~/openwiki-hub/link.sh'
alias owhu-stop='_owhu_stop'

_OWHUB_PID="/tmp/openwiki-hub.pid"

_owhu_run() {
  if [[ -f "$_OWHUB_PID" ]] && kill -0 "$(cat "$_OWHUB_PID")" 2>/dev/null; then
    kill "$(cat "$_OWHUB_PID")" 2>/dev/null
    rm -f "$_OWHUB_PID"
  fi
  ~/openwiki-hub/link.sh
  cd ~/openwiki-hub && python3 -m http.server 8765 --bind 127.0.0.1 >/tmp/openwiki-hub
  local pid=$!
  echo "$pid" > "$_OWHUB_PID"
  sleep 0.5
  if kill -0 "$pid" 2>/dev/null; then
    echo "🚀 OpenWiki Hub → http://localhost:8765"
    command -v open >/dev/null && open "http://localhost:8765" >/dev/null 2>&1
  else
    echo "❌ no arrancó. log:"
    tail -20 /tmp/openwiki-hub.log
  fi
}

_owhu_stop() {
  if [[ -f "$_OWHUB_PID" ]] && kill -0 "$(cat "$_OWHUB_PID")" 2>/dev/null; then
    kill "$(cat "$_OWHUB_PID")" 2>/dev/null
    rm -f "$_OWHUB_PID"
    echo "🛑 Hub parado"
  else
    echo "no hay hub corriendo"
  fi
}

```

Aplicar:

```
source ~/.zshrc
```

En bash cambia `~/.zshrc` por `~/.bashrc` y la cabecera del bloque.

6. Uso día a día

```
# Generar la wiki inicial de un proyecto nuevo (1ª vez)
cd mi-proyecto
git init                                # el hook se instala solo
owi                                     # ~1-3 min según tamaño del repo

# Actualizar la wiki manualmente cuando quieras
owu

# Ver TODAS tus wikis del sistema en una sola web
owhu                                    # abre http://localhost:8765

# Parar el hub
owhu-stop

# Re-escanear (si creas openwiki/ en otros proyectos)
owhu-link                               # solo reescanea, no sirve
owhu                                    # reescanea + sirve
```

Flujo típico con commits

```
cd mi-proyecto
# ... trabajas en código ...
git add .
git commit -m "feat: nueva feature"
# → el hook regenera openwiki/ (sin auto-add)
# → ves en `git status`:
#     M openwiki/quickstart.md
#     M openwiki/architecture.md
git diff openwiki/                      # revisas lo que cambió
git add openwiki/                       # si quieres incluirlo en ESTE commit
git commit --amend --no-edit
# o en uno aparte:
# git commit -m "docs: refresh wiki"
```

7. Privacidad y qué se envía a MiniMax

Qué	Dónde va	Notas
Tu API key	<code>~/.openwiki/.env</code> (chmod 600)	Nunca en el repo
Código fuente del proyecto	Servidor MiniMax (<code>api.minimax.io/anthropic</code>)	OpenWiki tiene que leerlo para documentarlo
Wiki generada (<code>openwiki/*.md</code>)	Carpeta local + opcionalmente al repo	Tú decides si la commiteas
Lista de archivos symlinked en el hub	Solo local	No sale de tu Mac

Si necesitas 100% on-prem, openwiki no soporta modelos locales out-of-the-box. Habría que forkear y enchufar Ollama/vLLM por el provider `openai-compatible`.

8. Troubleshooting

`openwiki --init` falla con `Invalid model ID configured in OPENWIKI_MODEL_ID`

El regex interno de openwiki rechaza corchetes. Usa `MiniMax-M3` (200k contexto) en vez de `MiniMax-M3[1m]` (1M contexto).

El hook no se ejecuta tras `git init`

```
ls .git/hooks/pre-commit          # debe existir
cat ~/.git-templates/hooks/pre-commit  # debe tener contenido
git config --global --get init.templateDir  # debe apuntar a ~/.git-templates
```

Si `git config` está vacío:

```
git config --global init.templateDir ~/.git-templates
```

El hub muestra "manifest fetch failed (404)"

El servidor está sirviendo pero falta `wikis.json`. Ejecuta:

```
owhu-link
```

```
# regenera symlinks + manifest
```

Los diagramas Mermaid no se renderizan

Comprueba la consola del navegador (F12 → Console). Si dice `mermaid is not defined`, el CDN falló (¿estás offline?). Solución permanente: descarga mermaid local:

```
curl -L https://cdn.jsdelivr.net/npm/mermaid@10.9.1/dist/mermaid.min.js \
  -o ~/openwiki-hub/mermaid.min.js
# En index.html, cambia el <script src=...> por:
# <script defer src="mermaid.min.js"></script>
```

Puerto 8765 ocupado

Cambia el puerto en dos sitios:

1. `~/openwiki-hub/link.sh` → no toca puerto
2. La línea `python3 -m http.server 8765` dentro de `_owhu_run()` en `~/zshrc`

9. Resumen de archivos creados

<code>~/openwiki/.env</code>	<code># config MiniMax (chmod 600)</code>
<code>~/git-templates/hooks/pre-commit</code>	<code># hook global (chmod +x)</code>
<code>~/openwiki-hub/index.html</code>	<code># UI del hub</code>
<code>~/openwiki-hub/styles.css</code>	<code># tema oscuro</code>
<code>~/openwiki-hub/app.js</code>	<code># carga markdown + mermaid</code>
<code>~/openwiki-hub/link.sh</code>	<code># scan + symlinks + manifest (chmod +x)</code>
<code>~/openwiki-hub/wikis/</code>	<code># symlinks a openwiki/ reales</code>
<code>~/openwiki-hub/wikis.json</code>	<code># manifest estático</code>
<code>~/zshrc</code>	<code># aliases + funciones añadidas al final</code>

Tras instalar todo, `owhu` desde cualquier directorio te abre `http://localhost:8765` con todas tus wikis en una sola web.